

SOFTWARE REQUIREMENTS SPECIFICATION

Charity Management System | Single-charity web application

Document field	Current value
Version	2.0 - implementation-aligned revision
Date	15 September 2026
Basis	July 2026 SRS v1.0 and repository implementation reviewed on 15 September 2026
Technology	Java 17, Spring Boot 4.0.3, Spring Security, Thymeleaf, PostgreSQL, Docker
Deployment	Render web service with Neon PostgreSQL; environment-based configuration
Status	Current implementation and remaining requirements; deployment-dependent controls require verification

Purpose

This revision describes the application as implemented so far, updates the functional scope and data model, and separates existing behavior from requirements that remain open. It is a requirements document, not a claim that every item has passed production acceptance.

1. Introduction

1.1 Purpose and scope

The system centralizes operational and financial records for one charity. Authenticated users work with members, years, budgets, donations, projects, events, annual membership receipts and reports. A public homepage shows only intentionally published impact projects and upcoming events.

1.2 Objectives

- Maintain consistent charity records and year-based summaries.
- Enforce URL-level and method-level role permissions.
- Provide search, filtering, pagination, PDF reports and selected Excel exports.
- Track annual membership receipts separately from donations.
- Limit abusive requests before they reach authentication or data-changing controllers.

1.3 Terms

Term	Meaning
RBAC	Role-based access control.
Membership year	Calendar year covered by an offline membership payment; independent of permission role.
Published project	Finished project approved by HEAD for the public homepage.
Public event	Upcoming event approved by HEAD for public display.
Void receipt	Historical membership receipt retained with correction reason and actor; excluded from paid totals.

2. Overall description

2.1 Architecture and operation

The application uses Spring MVC controllers, Thymeleaf templates, DTOs and mappers, service implementations, Spring Data JPA repositories, Hibernate entities and PostgreSQL. Docker builds and runs the Java 17 application. Database, administrator and other deployment settings are supplied through environment variables.

2.2 Current functions

- Registration, login, logout, profiles and role-sensitive navigation.
- CRUD and year grouping for the core operational modules.
- Membership fee configuration, receipt recording, voiding and history.
- Dashboard financial and activity totals, budget warnings and year summaries.
- PDF reports for years and related modules; Excel exports for filtered members and year donations.
- Public impact homepage with published finished projects, upcoming approved events and charts.
- Application logging, server-side validation, global error handling and request throttling.

2.3 Constraints

The application serves one charity. Public registration is currently enabled; the v1.0 requirement to disable it by deployment configuration is still open. Request limits are held in process memory and apply independently per running instance.

3. Users, roles and permissions

The Role enum currently defines HEAD, SUBHEAD, TREASURER, EVENT_MANAGER, PROJECT_MANAGER, VOLUNTEER, MEMBER, DONOR and SPONSOR. The exact rights are defined by Spring Security URL matchers and controller/service annotations; role names alone do not confer broad rights.

Capability	Observed permission rule
Manage years	HEAD creates, edits and deletes; HEAD/SUBHEAD/TREASURER/MEMBER may view.
Manage budgets	HEAD/TREASURER perform writes; broader core roles may view.
Manage memberships	HEAD/SUBHEAD/TREASURER access the ledger and record, void or set fees.
Publish impact/events	HEAD only.
Generate PDF reports	HEAD/SUBHEAD on the PDF controller.
Generate Excel exports	HEAD/SUBHEAD/TREASURER/MEMBER on the Excel controller.
Public homepage	Anonymous GET / is permitted.

Some controller rules for specialized roles and member editing differ by operation. Permission acceptance must therefore verify each direct endpoint; the simplified four-role matrix in v1.0 is no longer authoritative.

4. Functional requirements

4.1 Identity and profiles

ID	Requirement and current state
FR-AUTH-01	Authenticate by form login with BCrypt-backed credentials; role permissions apply to protected routes. Implemented.
FR-AUTH-02	Provide registration and member/account synchronization, plus profile viewing and permitted editing. Implemented.
FR-AUTH-03	Show access-denied and global error pages. Implemented.
FR-AUTH-04	Allow production registration to be disabled by configuration. Open; registration route is currently public.
FR-AUTH-05	Limit POST /login to 10 attempts and POST /register to 5 attempts per client per minute. Implemented in-process.

4.2 Core records

FR-CORE-01 - Authorized users shall create, list, view, update and delete members, years, budgets, donations, projects and events according to their individual endpoint permissions. Existing controllers and services implement these workflows. Member participation uses JPA relationships; years group related operational records.

FR-CORE-02 - Lists shall support the available entity-specific search, filters and pagination. Year views shall show financial and activity summaries. Existing dashboards and year summaries include aggregate totals and budget warnings.

FR-CORE-03 - The system shall validate form input, surface errors and preserve database relationships during edits and deletion. Existing validation and exception handling are present; scenario-specific integrity checks remain part of acceptance testing.

4.3 Membership dues

ID	Requirement and current state
FR-MSHIP-01	Keep permission role separate from annual payment status. Implemented.
FR-MSHIP-02	Allow HEAD/SUBHEAD/TREASURER to set a fee per year and record offline payments for past, current or future coverage years. Implemented; default EUR 10.00 is configurable.
FR-MSHIP-03	Keep original amount and name snapshot; void with reason rather than delete; prevent duplicate active receipts for member/year. Implemented.
FR-MSHIP-04	Show paid status on member lists, payment history on profiles, and membership income in dashboard/year/PDF summaries. Implemented.

4.4 Public impact and reporting

ID	Requirement and current state
FR-PUB-01	Anonymous homepage shows only HEAD-published finished projects and HEAD-published upcoming events. Implemented.
FR-PUB-02	HEAD publication shall validate finished project status or an event purpose and scheduled date. Implemented.
FR-RPT-01	Generate selected PDF reports, including complete-year totals with membership income. Implemented.
FR-RPT-02	Export filtered member data and donations for a selected year as XLSX. Implemented.
FR-RPT-03	Use no-store response caching for sensitive XLSX downloads. Implemented.

4.5 Request protection

FR-RATE-01 - Every request, including static assets and authentication routes, shall be subject to a 300-request per-client one-minute quota. POST /login: 10; POST /register: 5; other data-changing methods: 60. Exceeding a quota returns HTTP 429 and Retry-After: 60. Implemented with a bounded in-memory map.

FR-RATE-02 - Client identity shall use the direct peer address by default. X-Forwarded-For shall be used only when RATE_LIMIT_TRUST_PROXY=true and the application is reachable through a trusted proxy. Implemented as configuration; Render production setting must be verified.

5. External interfaces

The presentation is a responsive Thymeleaf/Bootstrap interface with forms, tables, dashboard, profile views, reports and a public landing page. Database access uses a PostgreSQL JDBC connection from environment variables. PDF and XLSX are downloadable HTTP responses. Production HTTPS and secure provider-supported database connections remain deployment acceptance conditions.

6. Data requirements

Entity	Current data/relationships
UserAccount / Member	Authentication identity and charity participant; profiles are synchronized with member information.
Years	Annual container for budgets, donations, projects and events.
Budget / Donation	Financial records linked to years; amounts currently stored as Double.
Project / Event	Year-linked work and activity, member assignments, and public visibility flags.
MembershipFee	BigDecimal amount keyed by membership year.
MembershipPayment	BigDecimal historical receipt, member ID/name snapshot, year, dates, actor, active key and optional void reason.

The activeKey unique constraint permits one active membership receipt for a member/year while retaining voided receipts. A deleted member does not erase a receipt name snapshot. Membership receipts are a separate ledger and must not be duplicated as donations.

Open data gap: the v1.0 SRS requires fixed-precision decimals for all money fields. Budget, donation and project amounts still use Double and should be migrated to BigDecimal/appropriate SQL numeric columns before claiming that requirement is met.

7. Non-functional requirements

ID	Requirement / verification
NFR-SEC-01	Authentication, role authorization and server-side validation for protected operations; verify direct HTTP access by role.
NFR-SEC-02	Rate limits reduce automated abuse. They do not prevent unauthorized database access, credential exposure or large distributed floods.
NFR-SEC-03	Protect credentials using environment variables, HTTPS and provider-supported database encryption; verify deployed configuration.
NFR-DATA-01	Migrate remaining floating-point money fields to fixed precision; open.
NFR-PERF-01	Use server-side pagination and aggregate queries for dashboard totals and budget alerts; performance must be measured under load.
NFR-REL-01	Membership receipt recording uses locking and a unique active key; external database must retain data across container restarts.
NFR-OBS-01	Logging and global exception handling support diagnosis without intentionally exposing secrets.
NFR-USA-01	Responsive, accessible navigation and form feedback remain subject to browser/accessibility review.

8. Business rules

- BR-01 - Protected modules require authenticated, authorized users; public GET / is the explicit exception.
- BR-02 - Permission roles and paid membership status are independent.

- BR-03 - One active membership payment per member/year; correction voids a receipt with reason and keeps history.
- BR-04 - Fee changes apply to new receipts only, and no future payment date may be recorded.
- BR-05 - Only HEAD publishes finished projects or suitably scheduled events.
- BR-06 - Reports shall use persisted data and include membership receipts only once.
- BR-07 - Excess requests are rejected before downstream authentication/controllers run.

9. Representative use cases

Use case	Actor	Expected result
UC-01 Login	Registered user	Role-appropriate authenticated session or generic failed-login response; quota applies.
UC-02 Record membership	Finance roles	Offline receipt stored with fee snapshot and member name.
UC-03 Correct receipt	Finance roles	Receipt voided with reason, retained in history; replacement may be recorded.
UC-04 Publish impact	HEAD	Finished project or valid upcoming event appears on public homepage.
UC-05 Export year report	HEAD/SUBHEAD	PDF includes annual records and membership income.
UC-06 Export spreadsheet	Authorized core role	Filtered members or year donations downloaded as XLSX.
UC-07 Flood requests	Any client	After quota, HTTP 429 with Retry-After; request does not reach data action.

10. Acceptance criteria and open work

- Verify all role-sensitive endpoints with direct HTTP requests and UI controls.
- Check CRUD integrity, year-linked summaries, filter preservation, and PDF/XLSX contents against persisted values.
- Verify membership duplicate prevention, void history and survivor snapshots after member deletion.
- Verify public publication rules and anonymous visibility.
- Verify quotas, proxy address configuration and expected 429 response on Render.
- Build Docker image, confirm Render/Neon connectivity and HTTPS, and review credential handling.
- Complete fixed-precision migration for remaining money fields.
- Add a production registration disable switch if that v1.0 requirement remains desired.
- Evaluate shared or edge rate limiting for multiple instances and large traffic volumes.

Repository tests cover memberships, PDF, Excel, profiles, public pages, budget warnings, dashboard queries, registration validation and the request limiter. A full local test run requires database environment variables for the general application-context test; absence of those variables caused one context-startup error during this review.

11. Logical relationship overview

UserAccount is linked to Member/profile data. Years groups Budget, Donation, Project and Event. Member participates in Project and Event and may be associated with Donation. MembershipFee is keyed by membershipYear; MembershipPayment stores a scalar memberId and name snapshot so it survives member deletion. Project and Event each have an opt-in public flag. This narrative replaces the v1.0 logical ER figure, which omitted membership and public visibility fields; physical JPA mappings remain the authoritative schema.

12. Assumptions and future scope

The system serves one charity and records offline membership payments; it does not process card charges. Future work may include multi-charity tenancy, OAuth/OIDC, notifications, receipt storage, public donation payments, external accounting integrations, financial approval history, shared rate-limit storage and fixed-precision migration. Spreadsheet export and dashboard/public analytics are now implemented and are no longer future-scope items.